
Spatial Atlas: Compute-Grounded Reasoning for Spatial-Aware Research Agent Benchmarks

Arun Sharma

University of Minnesota, Twin Cities
arunshar@umn.edu

Abstract

We introduce *compute-grounded reasoning* (CGR), a design paradigm for spatial-aware research agents in which answerable sub-problems are computed from explicit intermediate representations before a language model generates a response. Spatial Atlas instantiates CGR as an Agent-to-Agent (A2A) server with spatial question-answering and machine-learning engineering handlers. A structured spatial scene graph computes relationships from extracted entities. The ML path supports strategy-aware code generation, validation-score parsing, bounded refinement, and leak-audit prompts, but generated-code execution is disabled by default and requires explicit opt-in inside an isolated, trusted worker. This paper describes the implemented architecture and its evaluation protocol. It does not report FieldWorkArena results because the benchmark data were not accessible, and it intentionally omits performance, cost, and latency numbers that are not backed by reproducible run artifacts.

1 Introduction

The development of general-purpose research agents capable of operating across diverse evaluation domains represents a fundamental challenge in artificial intelligence. While large language models (LLMs) have demonstrated remarkable reasoning capabilities [1, 19], deploying them as autonomous agents that can reliably solve real-world tasks remains an open problem [22]. Two recent benchmarks highlight complementary dimensions of this challenge: FieldWorkArena [7], which evaluates multimodal spatial reasoning in industrial environments such as factories, warehouses, and retail spaces, and MLE-Bench [3], which tests end-to-end machine learning engineering across 75 Kaggle competitions.

Most existing agent architectures treat these benchmarks as independent problems, developing specialized systems for each [11, 26]. This fragmentation wastes shared infrastructure and misses opportunities for architectural insights that transfer across domains. For instance, the structured reasoning required to answer spatial questions (“How many pallets are within 3 meters of the emergency exit?”) shares fundamental properties with the systematic hypothesis testing needed to select effective ML strategies (“Which feature engineering approach maximizes validation accuracy for this tabular dataset?”).

We present **Spatial Atlas**, a spatial-aware research agent that exposes spatial question-answering and ML-engineering handlers through a single Agent-to-Agent (A2A) protocol server [8]. FieldWorkArena motivated the original spatial adapter, but the benchmark is not part of the reported evaluation because its data were not accessible. The system is organized around a design paradigm we call *compute-grounded reasoning* (CGR): wherever a sub-problem admits a deterministic solution, compute the answer first and supply it as a fact to the language model rather than asking the model to generate it. Our architecture instantiates CGR through five key contributions:

1. **Spatial Scene Graph Engine:** A structured representation that extracts entities and relations from vision model descriptions, computes spatial relationships, and exposes the provenance of those computations. Perception errors can still propagate into the graph.

2. **Entropy-Guided Reasoning:** An information-theoretic routing policy that estimates information gain for candidate actions and triggers reflection when confidence is low. Its accuracy and cost effects remain evaluation questions.
3. **Fail-Closed ML Pipeline:** A strategy-aware code generation system whose execution path requires execution opt-in, isolated-worker attestation, and authenticated server startup. An enabled run permits at most 3 total attempts with a 600-second timeout per attempt; dummy submissions require a separate opt-in.
4. **Score-Driven Refinement:** An iterative loop that parses machine-readable validation scores, asks the configured strong tier for a revision, and retains the revision only when its parsed score is better.
5. **Leak Audit Registry:** A prompt-based framework that asks generated pipelines to check common train/test leakage patterns and can inject task-specific hints.

The unifying principle behind these contributions is compute-grounded reasoning: wherever possible, we compute answers from structured representations rather than asking language models to generate them directly. This design improves inspectability because intermediate inputs and computations can be audited. Reliability, accuracy, latency, and cost must still be established through the planned evaluation.

2 Related Work

Agent Frameworks: The rapid development of LLM-based agent frameworks has produced systems spanning general-purpose reasoning and specialized domains. AutoGPT [21] pioneered autonomous LLM agents with self-directed task decomposition, while OpenDevin (now OpenHands) [11] established a software development agent framework with sandboxed code execution. SWE-Bench agents [13] demonstrated that LLMs can resolve real-world GitHub issues, and DAMO MLE-Agent [27] specifically targets Kaggle-style ML competitions. Our work differs in unifying two distinct benchmark domains under a single architecture with shared compute-grounded reasoning infrastructure.

Spatial Reasoning in Vision-Language Models: Vision-language models (VLMs) exhibit well-documented weaknesses in spatial reasoning tasks, particularly object counting, distance estimation, and relative positioning [4, 18]. Studies have shown that VLMs frequently hallucinate spatial relationships when asked to reason about complex scenes [16]. SpatialVLM [4] attempts to address this through specialized spatial training data. Our approach instead moves relationship computation into an explicit representation, while remaining dependent on the accuracy of entity extraction and geometric measurement.

Scene Graphs for Visual Reasoning: Scene graph representations, popularized by Visual Genome [15] and the GQA dataset [12], provide structured representations of visual scenes as graphs of objects and relationships. Neural scene graph generation [25] and scene graph-based visual question answering [9] have shown that explicit structure improves reasoning over raw visual features. Our spatial scene graph engine adapts these ideas to industrial environments, incorporating distance computation and constraint checking as first-class operations.

AutoML and Competition-Oriented Systems: Automated machine learning frameworks such as AutoGluon [5], Auto-sklearn [6], and AutoKeras [14] aim to automate the end-to-end ML pipeline. More recent work leverages LLMs for ML code generation [10], combining the flexibility of natural language understanding with systematic hyperparameter search. Our ML path adds strategy-aware code generation and bounded repair attempts behind a fail-closed execution gate.

A2A Protocol and Agent Interoperability: Google’s Agent-to-Agent (A2A) protocol [8] defines a standard for inter-agent communication, enabling heterogeneous agents to collaborate through a common interface. Our system implements a compliant A2A server that exposes both spatial reasoning and ML pipeline capabilities through a unified task interface, demonstrating the protocol’s flexibility for multi-domain agent deployment.

Information-Theoretic Reasoning: Active learning [20] and Bayesian experimental design [2] provide principled frameworks for selecting actions that maximize information gain. Recent work has applied these ideas to LLM reasoning chains [24], using uncertainty estimates to guide when to seek additional information. Our entropy-guided reasoning extends this paradigm to agent action selection, estimating which reasoning step will most reduce uncertainty about the final answer.

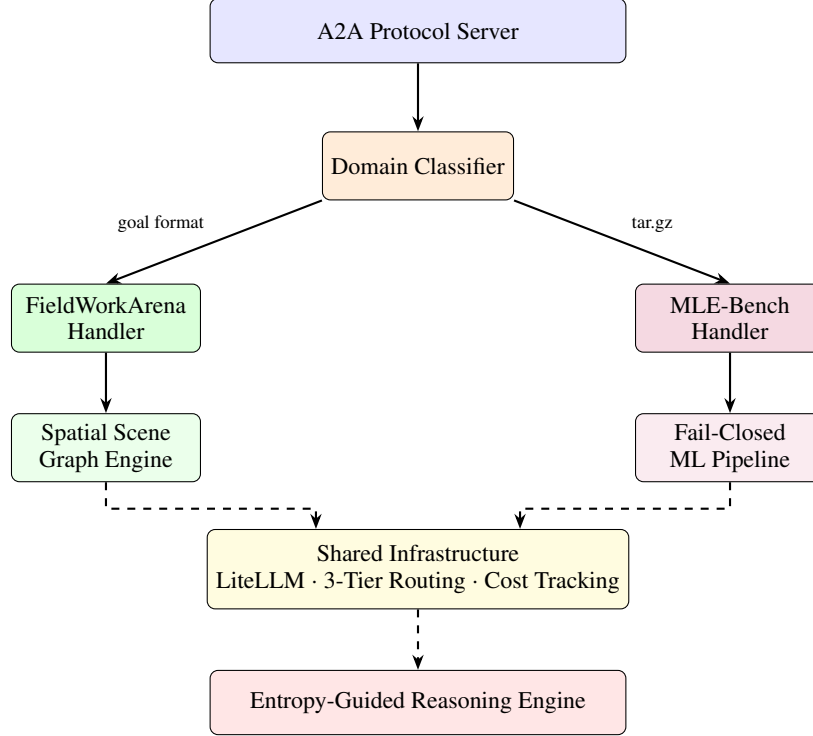


Figure 1: Spatial Atlas system architecture. The A2A server routes incoming tasks to domain-specific handlers through a classifier. Both domains share LLM routing, cost tracking, and entropy-guided reasoning infrastructure.

3 System Architecture

Spatial Atlas operates as a spatial-aware research agent exposed via a dual-domain A2A server. It receives task requests through a standardized protocol and routes them to the appropriate processing pipeline. Figure 1 illustrates the overall system design.

Domain Classification: The domain classifier operates on task metadata and attachment types. The FieldWorkArena adapter recognizes its documented goal shape, while MLE-Bench tasks arrive with `tar.gz` attachments containing competition datasets and description files. Classification uses deterministic rules rather than an LLM call. The implementation has not been benchmarked for routing latency or cost.

3.1 Shared Infrastructure

Both domain handlers share several critical infrastructure components:

LiteLLM Multi-Provider Wrapper: We use LiteLLM [17] to abstract across multiple LLM providers and record provider-reported usage when available. Provider reports and local estimates are not treated as exact tokenizer-equivalent counts.

Three-Tier Frontier Model Routing: We define three model tiers, *fast*, *standard*, and *strong*, each mapped to a distinct model drawn from two frontier providers as shown in Table 1. The routing decision is based on task complexity, estimated by the entropy-guided reasoning engine (Section 5). The default configuration maps Standard and Strong to GPT-4.1. Operators can override `ATLAS_STRONG_MODEL` to test a different provider. Same-model and cross-provider refinement remain planned ablations; no accuracy or cost result is claimed here.

Public Agent Token Reservation: The public A2A Agent uses a concurrency-safe `BudgetedLLMClient` against the legacy configuration field `Config.max_tokens_per_task = 150000`. Before each provider call, it heuristically estimates prompt usage and reserves that estimate plus the allowed maximum completion under a lock. Concurrent calls within one A2A execution cannot oversubscribe the estimated reservation counter. A new execution receives a fresh counter, even for the same A2A task ID. This is not exact tokenizer accounting and not a hard

Table 1: Configured model tiers and intended roles. This is a design table, not a performance or cost result.

Tier	Model	Intended role
Fast	GPT-4.1-mini	Classification and simple extraction
Standard	GPT-4.1	Primary reasoning and code generation
Strong	GPT-4.1	Reflection and refinement

provider-token boundary, because provider tokenizers and image accounting vary. Provider-reported usage remains authoritative. Frozen benchmark drivers retain the base observational `LLMClient`; their batch usage is reconciled from immutable journals, terminal counters, and result artifacts.

4 Spatial Scene Graph Engine

The spatial scene graph engine is the cornerstone of the spatial question-answering path and the unexecuted FieldWorkArena adapter. It is designed to make spatial computations explicit, but it does not remove errors introduced by perception, depth estimation, object matching, or coordinate assumptions.

4.1 Problem Formulation

Given an image I of an industrial environment (factory, warehouse, or retail space) and a natural language question q , the task is to produce an answer a that may require counting objects, estimating distances, checking spatial containment, or verifying safety compliance. Directly prompting a VLM with (I, q) is unreliable because VLMs hallucinate spatial relationships and struggle with precise counting.

4.2 Scene Graph Construction

Our approach decomposes the problem into three stages: *extraction*, *structuring*, and *computation*.

Stage 1: Entity Extraction: We employ a two-pass extraction process. First, a vision-language model (GPT-4.1 with vision) generates a detailed textual description of the scene, prompted to enumerate all visible objects with approximate positions and attributes. Second, Florence-2 [23], a lightweight vision foundation model, performs object detection to obtain precise bounding boxes and counts, serving as a grounding mechanism for the VLM’s descriptions.

Stage 2: Graph Construction: Extracted entities are formalized as a spatial scene graph $G = (V, E)$ where vertices V represent entities and edges E represent spatial relations:

$$v_i = \text{SpatialEntity}(\text{id}_i, \text{label}_i, \text{pos}_i, \text{attrs}_i, \text{zone}_i) \quad (1)$$

$$e_{ij} = \text{SpatialRelation}(\text{subj}_i, \text{pred}_{ij}, \text{obj}_j, d_{ij}) \quad (2)$$

where $\text{pos}_i \in \mathbb{R}^2$ denotes the estimated position (from bounding box centroids), attrs_i is a dictionary of visual attributes (color, size, state), zone_i identifies the semantic zone (e.g., loading dock, aisle 3), and d_{ij} is the computed Euclidean distance between entities.

Stage 3: Deterministic Computation: The scene graph supports several query operations that produce verifiable facts:

- `query_near( $v, r$ )`: Returns all entities within radius r of entity v .
- `check_constraints( $C$ )`: Evaluates a set of spatial constraints C (e.g., minimum clearance distances) and returns violations.
- `count_by_label( $\ell$ )`: Returns the count of entities matching label ℓ , cross-referenced with Florence-2 detections.
- `to_fact_sheet()`: Serializes the graph into a structured natural language summary suitable for LLM consumption.

The fact sheet is then provided to the LLM alongside the original question, enabling it to answer based on computed facts rather than visual estimation.

Scoring Functions: The local FieldWorkArena adapter implements six scoring interfaces based on the available task specification. Because the gated benchmark data were not accessible, these interfaces have not been validated against an official FieldWorkArena evaluation run and are retained as dormant compatibility code.

Table 2: Local FieldWorkArena adapter scoring interfaces. The table documents code paths, not official benchmark validation.

Metric	Description
fuzzy_match	Token-level overlap with configurable threshold (default 0.8)
exact_match	Case-insensitive exact string equality
must_include	Predicted answer must contain all specified substrings
must_exclude	Predicted answer must not contain any specified substrings
json_match	Structured comparison of JSON objects with field-level matching
numerical_match	Numeric comparison with configurable tolerance ($\epsilon = 0.05$)

5 Entropy-Guided Reasoning

The entropy-guided reasoning engine provides a principled framework for selecting actions that maximize information gain while minimizing computational cost. This framework draws on active learning [20] and Bayesian experimental design [2], adapted to the sequential decision-making context of agent reasoning.

5.1 Information State Representation

At each reasoning step t , the agent maintains a knowledge state $\mathcal{K}_t$ consisting of accumulated observations, computed facts, and intermediate conclusions. We define the *answer entropy* as the uncertainty over the space of possible answers:

$$H(\mathcal{A} \mid \mathcal{K}_t) = - \sum_{a \in \mathcal{A}} P(a \mid \mathcal{K}_t) \log P(a \mid \mathcal{K}_t) \quad (3)$$

where $\mathcal{A}$ is the set of candidate answers and $P(a \mid \mathcal{K}_t)$ is the estimated probability of answer a given current knowledge.

5.2 Action Selection via Information Gain

Given a set of candidate actions $\{c_1, \dots, c_m\}$ (e.g., examining a specific region of the image, querying the scene graph, calling a stronger model), we select the action that maximizes expected information gain:

$$c^* = \arg \max_{c_j} \mathbb{E} [H(\mathcal{A} \mid \mathcal{K}_t) - H(\mathcal{A} \mid \mathcal{K}_t \cup \text{obs}(c_j))] \quad (4)$$

In practice, we approximate this using self-reported model confidence. Each candidate answer a is accompanied by a score $\sigma(a) \in [0, 1]$ from a prompting heuristic; this score has not been demonstrated to be calibrated.

5.3 Reflection and Confidence Thresholds

The entropy-guided system triggers a *reflection* step when the confidence score falls below a threshold:

$$\text{reflect}(a) = \begin{cases} \text{True} & \text{if } \sigma(a) < \tau \\ \text{False} & \text{otherwise} \end{cases} \quad (5)$$

where $\tau = 0.6$ is the reflection threshold. During reflection, the agent re-examines its reasoning with additional context (e.g., re-querying the scene graph with refined parameters, examining a different region of the image, or escalating to the strong model tier). A maximum of 2 reflection rounds is permitted per task to bound computational cost.

5.4 Cost-Efficiency Through Model Routing

The entropy framework informs model tier selection. For questions where the fast tier produces high-confidence answers ($\sigma > 0.8$), no escalation occurs. When confidence is moderate ($0.6 \leq \sigma \leq 0.8$), the standard tier is engaged. Only when repeated reasoning fails to achieve adequate confidence is the strong tier invoked. This progressive policy is intended to limit unnecessary escalation; its cost and quality effects remain part of the planned evaluation.

Algorithm 1 summarizes the complete entropy-guided reasoning procedure.

Algorithm 1 Entropy-Guided Reasoning

Require: Task T , knowledge state $\mathcal{K}_0$, budget B , threshold τ

```
1:  $a_0, \sigma_0 \leftarrow \text{FastModel}(T, \mathcal{K}_0)$ 
2: if  $\sigma_0 \geq 0.8$  then
3:   return  $a_0$ 
4: end if
5:  $\mathcal{K}_1 \leftarrow \mathcal{K}_0 \cup \text{SceneGraph}(T)$ 
6:  $a_1, \sigma_1 \leftarrow \text{StandardModel}(T, \mathcal{K}_1)$ 
7: for  $r = 1$  to 2 do
8:   if  $\sigma_1 \geq \tau$  then
9:     return  $a_1$ 
10:  end if
11:   $\mathcal{K}_{r+1} \leftarrow \text{Reflect}(\mathcal{K}_r, a_1)$ 
12:   $a_1, \sigma_1 \leftarrow \text{StrongModel}(T, \mathcal{K}_{r+1})$ 
13: end for
14: return  $a_1$ 
```

Table 3: ML strategy templates and their target competition types.

Strategy	Task Type	Key Components
Tabular	Classification/Regression	LightGBM/XGBoost, feature engineering, cross-validation
NLP	Text Classification/NER	Transformer fine-tuning, TF-IDF fallback
Vision	Image Classification	Pre-trained CNN, transfer learning, augmentation
TimeSeries	Forecasting	Prophet, ARIMA, lag features, rolling statistics
General	Mixed/Unknown	Ensemble of lightweight models
AutoGluon	Any (fallback)	Time-limited AutoGluon TabularPredictor

6 Fail-Closed ML Pipeline

The MLE-Bench handler generates candidate pipelines from competition descriptions. Execution is fail closed: it requires both `ATLAS_ENABLE_MLEBENCH_CODE_EXECUTION=true` and `ATLAS_TRUSTED_ISOLATED_WORKER=true`. Server startup then requires `ATLAS_BEARER_TOKEN` containing at least 32 characters.

Competition Analysis: Upon receiving a competition task, the analyzer extracts structured metadata including the task type, evaluation metric, data format, target column, and any special constraints. We classify competitions into six categories based on these features, as shown in Table 3.

6.1 Code Generation and Execution

For each competition, the pipeline generates a complete, self-contained Python script that:

1. Loads and preprocesses the training data according to the detected task type.
2. Implements the selected strategy with appropriate hyperparameters.
3. Trains the model with cross-validation when the generated strategy supports it.
4. Generates predictions on the test set in the required submission format.
5. Writes a valid `submission.csv` to the expected output location.

After explicit authorization, the generated script runs in a bounded subprocess with a 600-second timeout. The subprocess captures bounded stdout and stderr, uses a minimal environment, and is terminated as a process group on timeout or cancellation. These controls are defense in depth, not a complete security sandbox.

Bounded Repair Loop: When an explicitly authorized execution fails, the repair mechanism may:

1. **Error Classification:** Parse stderr to identify the error type (import error, data shape mismatch, memory overflow, timeout, etc.).

2. **Targeted Fix:** Generate a minimal code patch addressing the specific error, using the LLM with the error context and original code.
 3. **Re-execution:** Run the patched script with the same timeout constraints.
- `max_code_iterations = 3` means 3 total attempts, including the initial attempt. If all attempts fail, the task fails by default. A schema-shaped dummy submission is produced only when the operator separately sets `ATLAS_ALLOW_DUMMY_SUBMISSION=true`.

Score-Driven Refinement Loop: Error recovery alone cannot raise a working pipeline’s score; it only rescues pipelines that crash. After the first explicitly authorized run succeeds, the handler can parse a machine-readable line of the form `VALIDATION_SCORE: <float>`, request one targeted revision, re-run it under the same authorization and controls, and retain it only when the parsed score is better under the metric direction.

The loop runs up to `max_refinement_iterations = 2` extra passes, bounded by a hard wall-clock ceiling (`refinement_wall_time_seconds = 900`) to stay within MLE-Bench’s per-task budget. The selection logic is configured to discard revisions that regress or fail to print a score. This loop uses the configured Strong tier. Strong defaults to GPT-4.1 and can be overridden by the operator. Whether a cross-provider override outperforms a same-model retry is a planned ablation, not an established result.

6.2 Leak Audit and Targeted Leak Registry

The MLE-Bench paper and subsequent Kaggle post-mortems document a handful of competitions where the test set is reconstructable from training-set overlap, public dataset ancestry, or file metadata. Rather than hand-coding brittle exploit solvers (whose hard-coded merge keys may not match the MLE-Bench tar layout), Spatial Atlas maintains a *leak hint registry* whose entries are pure text instructions injected into the Strong-tier codegen prompt when a competition is detected. Every codegen call also receives a universal *leak audit preamble* that instructs the Strong model to, before training any model:

1. Compare ID-like columns between train and test for row-level overlap.
2. Compute row fingerprints (hash of non-target features) to detect content duplication.
3. Check temporal ordering for timestamp-based competitions (train/test leakage through temporal shuffling).
4. Hash file bytes for media-based competitions to detect identical test/train files.

The audit fires independently of any registered entry, so new or unregistered leaks are still caught as long as their exploit fits one of the four standard shapes. Registered entries carry competition-specific detection predicates and targeted exploit sketches that take precedence over the generic audit. This design keeps the exploit code adaptive: the Strong model writes the final pandas operations against the actual tar layout it sees at runtime, while the audit policy itself remains auditable in a single file (`mlebench/strategies/leaks.py`).

Strategy Selection via Entropy: The entropy-guided framework (Section 5) also informs strategy selection for ML competitions. When the competition description is ambiguous about the optimal approach, the system estimates confidence for each strategy template and may generate multiple candidate solutions, selecting the one with the highest validation score.

7 Implementation Details

A2A Protocol Compliance: Spatial Atlas implements the A2A protocol specification using the official `a2a-sdk` (version $\geq 0.3.20$). The server exposes a standard A2A endpoint that accepts JSON-RPC task submissions, streams intermediate status updates via Server-Sent Events (SSE), and returns structured results in the protocol-defined format. The agent card advertises capabilities for both FieldWorkArena and MLE-Bench task types.

Deployment: The system is packaged as a Docker container targeting `linux/amd64`. Environment variables configure API keys, model endpoints, and resource limits. Public HTTP request bodies default to a 64 MiB limit. Active requests default to a maximum of 4; excess requests receive HTTP 503 rather than entering an unbounded queue. `ATLAS_BEARER_TOKEN`, when configured, must contain at least 32 characters and authenticates non-read-only requests. Normal non-loopback startup requires the token even when MLE execution is disabled; loopback development may omit it. `ATLAS_ALLOW_UNAUTHENTICATED_PUBLIC=true` is a test-only override for non-loopback binding and is not a normal deployment mode. MLE execution always requires authenticated startup.

File Processing Pipeline: Task inputs arrive in diverse formats requiring specialized processing:

- **Images:** JPEG/PNG files are processed through both GPT-4.1 vision (for scene description) and Florence-2 (for object detection and counting). Images are resized to a maximum of 1568 pixels on the longest edge to manage API costs.
- **PDFs:** Extracted using `pypdf` with page-by-page text extraction and optional OCR fallback.
- **Videos:** Frame extraction via OpenCV at 1 FPS, with keyframe selection based on scene change.
- **Archives:** `tar.gz` files (MLE-Bench data) are extracted to a temporary workspace directory.
- **Text:** Direct UTF-8 processing with encoding detection fallback.

Model Configuration: All LLM calls use the model configurations specified in Table 1. The fast tier (`openai/gpt-4.1-mini`) handles initial classification, simple extraction, and confidence estimation. The standard and strong tiers default to `openai/gpt-4.1`; Strong can be overridden with `ATLAS_STRONG_MODEL`. Any cross-provider variant must be compared with the default under identical budgets.

Resource Controls: Public A2A model calls use the heuristic concurrency-safe per-execution reservation described above. Frozen benchmark drivers remain observational and use artifact-based batch accounting. Reflection is limited to a maximum of 2 rounds per task. Generated-code execution is disabled by default and requires both execution flags plus authenticated server startup. After authorization, each pipeline attempt has a 600-second timeout and the initial plus repair loop permits at most 3 total attempts. Dummy submissions remain disabled unless separately enabled. After a successful real run, the score-driven refinement loop may execute up to 2 additional passes (Section 6), bounded by a 900-second wall-clock ceiling.

8 Evaluation

Current Evidence Boundary: No claim-bearing benchmark table is reported in this version. FieldWorkArena remained gated and inaccessible, so the project did not run its validation set and reports no FieldWorkArena accuracy, ablation, latency, token, or cost result. The local adapter and tests establish software behavior only; they are not benchmark evidence.

The repository also does not contain a sealed, end-to-end MLE-Bench result artifact covering the full competition suite. Accordingly, previously drafted valid-submission, medal, refinement, leak-effectiveness, and cost figures have been removed. A completed job or a working code path is not treated as a scientific result without the corresponding immutable predictions, scorer output, run manifest, and logs.

Planned Evaluation Protocol: The spatial evaluation will compare a question-only baseline, the scene-graph path, the metric-perception path, and a native reference implementation on a frozen public slice. It will report per-question-type accuracy, paired uncertainty intervals, parser and geometry failure rates, model and data revisions, token usage, wall-clock latency, and artifact paths. Labels remain sealed until all prediction journals are complete.

The ML-engineering evaluation will run fixed competition subsets with identical budgets and report valid-submission rate, competition-specific score, refinement acceptance rate, execution failures, tokens, latency, and cost. Each aggregate must be generated from machine-readable run artifacts rather than copied into the paper manually.

9 Discussion

Limitations: Several limitations merit discussion. First, the quality of spatial computation depends on perception, object correspondence, depth, scale, and coordinate conventions; deterministic arithmetic cannot correct an incorrect geometric input. Second, the FieldWorkArena adapter is unvalidated against the inaccessible benchmark and must not be presented as evaluated compatibility. Third, the ML pipeline’s strategy templates are hand-designed for common competition types, and novel tasks may fall outside their coverage. Fourth, cross-provider refinement and entropy-guided routing are design hypotheses whose accuracy and cost effects remain unmeasured. Fifth, the leak audit covers only four common leakage shapes and may miss leakage in non-standard metadata or file formats. Finally, generated code requires isolation, resource limits, and explicit trust boundaries before production use. **Future Work:** Several directions for future work are promising:

- **Domain-Specific Fine-Tuning:** Fine-tuning Florence-2 on industrial environment imagery could significantly improve object detection accuracy, particularly for domain-specific objects like safety equipment, pallet types, and industrial signage.
- **Multi-Agent Collaboration:** The A2A protocol enables multi-agent architectures where specialized sub-agents handle specific sub-tasks (e.g., one agent for visual analysis, another for spatial computation, a third for language generation).

- **Streaming Responses:** Implementing streaming A2A responses would enable real-time feedback during long-running ML pipeline executions.
- **Expanded Benchmarks:** Extending the architecture to additional benchmarks (e.g., SWE-Bench for software engineering, WebArena for web navigation) would test the generality of our approach.

Broader Impact: The spatial scene graph approach has direct applications to industrial safety, where automated monitoring of safety compliance (clearance distances, equipment placement, emergency exit accessibility) could prevent workplace injuries. However, automated spatial reasoning systems must be deployed carefully, with human oversight, as errors in safety-critical applications could have severe consequences.

10 Conclusion

We have presented Spatial Atlas, a spatial-aware research agent built on the compute-grounded reasoning (CGR) paradigm and exposed through an A2A protocol server. Our implemented contributions are:

1. A **spatial scene graph engine** that makes extracted entities, coordinate assumptions, and computed relationships inspectable without claiming that deterministic computation eliminates perception errors.
2. An **entropy-guided reasoning framework** for model routing and targeted reflection, with effects to be measured under the planned ablations.
3. A **fail-closed ML pipeline** with strategy-aware code generation, explicit execution authorization, bounded repair attempts, and a separately gated dummy fallback.
4. A **score-driven refinement loop** that parses validation scores, requests a revision from the configured strong tier, and retains it only when its parsed score improves.
5. A **leak audit registry** that prompts generated pipelines to check four common leakage patterns and can inject task-specific hints.

Compute-grounded reasoning, the principle of computing what can be computed before generating what must be generated, offers a design pattern for making agent decisions more inspectable. The planned evaluations will determine when this additional structure improves accuracy, reliability, latency, or cost.

Spatial Atlas is open-sourced at <https://github.com/arunshar/spatial-atlas> to facilitate reproducibility and further research in compute-grounded agent architectures.

References

- [1] Anthropic. Claude model family: Claude Opus 4.6 and Claude Sonnet 4.6. *Technical report*, 2025.
- [2] K. Chaloner and I. Verdinelli. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304, 1995.
- [3] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, 2025. arXiv:2410.07095.
- [4] B. Chen, Z. Xu, S. Kirmani, et al. SpatialVLM: Endowing vision-language models with spatial reasoning capabilities. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [5] N. Erickson, J. Mueller, A. Shirkov, et al. AutoGluon-Tabular: Robust and accurate AutoML for structured data. *arXiv preprint arXiv:2003.06505*, 2020.
- [6] Matthias Feurer, Aaron Klein, Katharina Eggensperger, et al. Auto-sklearn 2.0: Hands-free AutoML via meta-learning. *Journal of Machine Learning Research*, 22(235):1–61, 2019.
- [7] J. Takahashi, A. Moteki, A. Uchida, S. Masui, F. Yang, K. Uchino, Y. Song, Y. Bisk, G. Neubig, I. Kusajima, Y. Watanabe, H. Ishida, K. Nakagawa, and S. Jiang. FieldWorkArena: Agentic AI benchmark for real field work tasks. *arXiv preprint arXiv:2505.19662*, 2025.
- [8] Google. Agent-to-Agent (A2A) protocol specification. Online documentation, 2024.
- [9] M. Hildebrandt, H. Li, R. Koner, et al. Scene graph reasoning for visual question answering. *arXiv preprint arXiv:2007.01072*, 2020.
- [10] Nikolaus Hollmann, Stefan Müller, and Frank Hutter. Large language models for automated machine learning. *arXiv preprint arXiv:2402.00878*, 2024.
- [11] S. Hong, X. Wang, J. Yu, et al. OpenDevin: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- [12] Drew Hudson and Christopher Manning. GQA: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.
- [13] Carlos Jimenez, John Yang, Ananya Wettig, et al. SWE-Bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024.

- [14] Haifeng Jin, Qingquan Song, and Xia Hu. AutoKeras: An AutoML library for deep learning. *Journal of Machine Learning Research*, 24(6):1–6, 2023.
- [15] Ranjay Krishna, Yuke Zhu, Oliver Groth, et al. Visual Genome: Connecting language and vision using crowdsourced dense image annotations. *International Journal of Computer Vision*, 123:32–73, 2017.
- [16] Y. Li, Y. Du, K. Zhou, et al. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [17] BerriAI. LiteLLM: Call 100+ LLM APIs using the OpenAI format. GitHub repository, 2024.
- [18] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2024.
- [19] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [20] Burr Settles. Active learning literature survey. *Computer Sciences Technical Report 1648*, University of Wisconsin–Madison, 2009.
- [21] Significant Gravitas. AutoGPT: An autonomous GPT-4 experiment. GitHub repository, 2023.
- [22] Lei Wang, Chunping Ma, Xinyi Feng, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):1–26, 2024.
- [23] B. Xiao, H. Wu, W. Xu, et al. Florence-2: Advancing a unified representation for a variety of vision tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [24] S. Xie, O. Levy, et al. Active prompting with chain-of-thought for large language models. *arXiv preprint arXiv:2302.12246*, 2024.
- [25] Danfei Xu, Yuke Zhu, Christopher Choy, and Li Fei-Fei. Scene graph generation by iterative message passing. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.
- [26] John Yang, Carlos Jimenez, Ananya Wettig, et al. SWE-Agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [27] Y. Zhang, H. Mao, Y. Zheng, et al. MLE-Agent: Automated machine learning engineering with LLM agents. *arXiv preprint arXiv:2402.15642*, 2024.